

FAST-MVH: K-d Tree Indexing for Dominance Checking in Multi-Objective Search

Abstract

Multi-objective shortest path search must maintain multiple Pareto-optimal solutions at each state. When heuristics take the form of multiple trade-off vectors, the cost of testing dominance can exceed the cost of generating new paths. This work reduces that cost through spatial indexing: truncated state frontiers are indexed with dynamic k-d trees; the full-dimensional fallback is confined to an append-only residue set indexed by a static k-d forest. Local dominance is tested before heuristic selection, avoiding expensive heuristic evaluations on doomed paths. On three to six objectives across grids and road networks, FAST-MVH returns identical solutions in $1.8\times$ to $82\times$ less time, reducing full-dimensional comparisons by 8–100 times.

1 Introduction

Multi-objective shortest path search differs from single-objective search in two ways. First, paths need not have a single cheapest cost; the algorithm must return one path for each distinct Pareto-optimal vector. Second, when individual lower-bound estimates are weak, using multiple estimates—a multi-valued heuristic—can prune more effectively than any single bound. But these benefits require testing dominance often: every new path candidate must be tested against existing solutions, and every heuristic vector must be tested against the goal frontier.

This work reduces the cost of dominance testing through spatial indexing. Truncated state frontiers are indexed with k-d trees, exploiting the fact that only the last $M - 1$ coordinates matter for local dominance comparison. When a truncated query finds only partial dominators, an append-only residue set of evicted vectors is searched in full dimensions, using a k-d forest built incrementally via the logarithmic method. Local dominance is tested before heuristic selection, avoiding heuristic evaluation on paths certain to be pruned.

The result is an algorithm called FAST-MVH that solves multi-objective shortest path problems with the same decisions and solutions as standard search but substantially faster. On instances from three to six objectives, speedups range from $1.8\times$ to $82\times$, with larger gains on problems having denser, more varied Pareto frontiers.

2 Problem Definitions

Let $G = (S, E, c)$ be a finite directed graph where each edge has cost $c(e) \in \mathbb{R}_{\geq 0}^M$. The cost of a path is the component-wise sum of edge costs. Vector x weakly dominates y (written $x \preceq y$) if $x_k \leq y_k$ for all $k \in [1, M]$. The task is to return all paths with distinct Pareto-optimal costs—costs not dominated by any other reachable cost.

The truncation operator drops the first coordinate:

$$\text{Tr}(x) = (x_2, \dots, x_M).$$

A **reduced frontier** $F(s)$ at state s is a set of cost vectors that maintains coverage: for every cost ever generated at s , some frontier member truncation-dominates it. When a new cost g is generated at s :

$$F(s) \leftarrow \{p \in F(s) : \text{Tr}(g) \not\preceq \text{Tr}(p)\} \cup \{g\}.$$

Frontiers need not be full-dimensional Pareto sets; they need only preserve coverage.

An **admissible multi-valued heuristic** assigns to each state an ordered finite set:

$$H(s) = \langle h^1, \dots, h^{k(s)} \rangle, \quad h^1 \leq_{\text{lex}} \dots \leq_{\text{lex}} h^{k(s)},$$

where for each goal-reaching suffix from s , some h^i bounds it. The lexicographic ordering is a correctness requirement: admissibility depends on selecting the first surviving vector in order.

The **goal frontier** is $T = F(s_{\text{goal}})$, the set of Pareto-optimal solution costs found so far. A cost vector f is **goal-dominated** if T contains a member

dominating $\text{Tr}(f)$. A cost vector g at state s is **locally dominated** if some previously expanded cost at s dominates g in all M coordinates.

3 Algorithm

3.1 Node lifecycle

FAST-MVH follows standard multi-objective search: extract a minimum- f node from OPEN; if goal-dominated, select a new heuristic and reinsert; otherwise expand it, generate successors, and test each for local dominance and heuristic acceptability before insertion.

A node records state $s(n)$, cost $g(n)$, heuristic index $i(n)$, and $f(n) = g(n) + h^{i(n)}$. The OPEN list is ordered lexicographically by f .

Algorithm 1: FAST-MVH Search

Input: Graph G , start state s_0 , goal state s_g , admissible multi-valued heuristics H .

Output: Pareto-optimal solution paths.

```

1  Sols  $\leftarrow \emptyset$ ;  $F(s), R(s) \leftarrow \emptyset$  for all  $s$ 
2   $n \leftarrow (s_0, \mathbf{0}, 1)$ ; OPEN  $\leftarrow \{n\}$ 
3  while OPEN  $\neq \emptyset$ 
4     $n \leftarrow \text{OPEN.PopMin}()$ 
5    if GoalDom( $f(n)$ ) then
6       $i \leftarrow \text{ChooseH}(s(n), g(n), i(n) + 1, \text{true})$ 
7      if  $i \neq \perp$  then  $i(n) \leftarrow i$ ; insert  $n$  in OPEN; continue
8    if LocalDom( $s(n), g(n)$ ) then continue
9    Update( $s(n), g(n)$ )
10   if  $s(n) = s_g$  then Sols  $\leftarrow$  Sols  $\cup \{n\}$ ; continue
11   for  $s' \in \text{Succ}(s(n))$ 
12      $g' \leftarrow g(n) + c(s(n), s')$ 
13     if LocalDom( $s', g'$ ) then continue
14      $i' \leftarrow \text{ChooseH}(s', g', 1, \text{false})$ 
15     if  $i' \neq \perp$  then insert  $(s', g', i')$  in OPEN
16 return Sols

```

3.2 Dominance queries

Both goal dominance and local dominance are queries on sets of vectors. FAST-MVH indexes both T and $F(s)$ with k-d trees over truncated coordinates.

A k-d tree node v stores a small array of vectors $P(v)$, bounding box $[\ell(v), u(v)]$ for its subtree, and two children. An unindexed array forms a single-node tree.

Procedure Find($\mathcal{K}, q, \kappa$) enumerates members p of forest $\mathcal{K}$ with $\kappa(p) \preceq q$, where κ is either Tr (truncated) or identity (full). It prunes a subtree at v if $\kappa(\ell(v)) \not\preceq q$, since no member can then dominate q . Callers typically stop at the first match, making pruning essential.

Algorithm 2: Dominance Queries

```

1 function Find( $\mathcal{K}, q, \kappa$ ) // yield members with  $\kappa(p) \leq q$ 
2    $\Sigma \leftarrow$  roots of  $\mathcal{K}$ 
3   while  $\Sigma \neq \emptyset$ 
4     pop  $v$ ; if  $\kappa(\ell(v)) \leq q$  then continue
5     yield each  $p \in P(v)$  with  $\kappa(p) \leq q$ 
6     push children of  $v$  onto  $\Sigma$ 
7 function GoalDom( $f$ )
8   return [Find( $T, \text{Tr}(f), \text{Tr}$ ) yields]
9 function LocalDom( $s, g$ )
10   $w \leftarrow \text{NONE}$ 
11  for  $p \in \text{Find}(F(s), \text{Tr}(g), \text{Tr})$ 
12    if  $p_1 \leq g_1$  then return true
13     $w \leftarrow \text{TR}$ 
14  if  $w = \text{NONE}$  then return false
15  return [Find( $R(s), g, \text{id}$ ) yields]
16 procedure Update( $s, g$ )
17  for  $p \in F(s)$  with  $\text{Tr}(g) \leq \text{Tr}(p)$ 
18    remove  $p$  from  $F(s)$ 
19    if  $p_1 < g_1$  then append  $p$  to  $R(s)$ 
20  insert  $g$  into  $F(s)$ 

```

LocalDom first searches $F(s)$ for truncated dominators (line 11). If a truncated dominator also dominates in the first coordinate (line 12), the path is locally dominated and pruned. If only truncated (not full) dominators exist ($w = \text{TR}$, line 13), the residue $R(s)$ is searched in all M coordinates (line 15). This avoids full-dimensional search through $F(s)$ when truncation suffices.

3.3 Residue set and indexed fallback

When Update evicts p from $F(s)$ due to $\text{Tr}(g) \leq \text{Tr}(p)$, two outcomes are possible: - $g \leq p$ fully: p is forever dominated; discard it. - $g_1 > p_1$: p may dominate future arrivals with smaller first coordinate; retain it.

FAST-MVH retains p in residue $R(s)$ iff $p_1 < g_1$. This set is append-only and potentially large, so indexing it efficiently is critical.

FAST-MVH uses the logarithmic method of Bentley and Saxe: new vectors append to an unindexed buffer. When the buffer reaches 64 vectors, it becomes an immutable k-d tree (contiguous, with medians positioned via radix permutation) and is locked. Subsequent buffers append, and when a predecessor buffer reaches size 64 and the current buffer exceeds 64, they merge into one larger tree. This ensures $O(\log|R(s)|)$ blocks and $O(\log|R(s)|)$ rebuild operations per appended vector.

3.4 Heuristic selection

ChooseH(s, g, j, b) returns the smallest index $i \geq j$ such that GoalDom($g + h^i$) is false, or $\perp$ if all are refuted.

To accelerate this, precompute per state the redundancy pointer:

$$\rho_s(i) = \max\{i' < i : \text{Tr}(h^{i'}) \leq \text{Tr}(h^i)\} \cup \{0\}.$$

If $\rho_s(i) \geq r$ (where r is the last-verified index), then h^i is refuted by the same goal vector that refuted $h^{\rho_s(i)}$, so skip it.

Algorithm 3: Heuristic Selection

```

1 function ChooseH( $s, g, j, b$ )
2   if  $j > k(s)$  then return  $\perp$ 
3   if  $T = \emptyset$  then return  $j$ 
4    $r \leftarrow (j - 1)$  if  $b$  else  $j$ 
5   for  $i = j$  to  $k(s)$ 
6     if  $\rho_s(i) \geq r$  then continue
7     if not GoalDom( $g + h^i$ ) then return  $i$ 
8   return  $\perp$ 

```

Correctness follows from $H(s)$'s lexicographic ordering and the fact that ChooseH returns the first surviving index.

4 Correctness

Lemma 1 (Residue sufficiency). If cost $a \in C(s)$ is not in $F(s)$ and no frontier member dominates a , then a full dominator of a exists iff a residue member dominates a .

Proof. Let $a_0 = a$. Since $a \notin F(s)$, some expansion b_0 evicted it. Either $b_0 \leq a_0$ (in which case any dominator is an evictor descendant), or $b_{0,1} > a_{0,1}$. In the latter case, follow the chain of evictors: $a_1 = b_0$, and let b_1 be the expansion evicting a_1 , and so on. Each step has $a_{i+1} \leq a_i$ (chain property). Since $a_{i+1,1}$ increases and $a_{i,1}$ decreases, the chain terminates. It cannot end in $F(s)$ by assumption, so it ends in $R(s)$. $\square$

Theorem 1 (Search equivalence). Under identical OPEN tie-breaking, successor order, and H ordering, FAST-MVH and standard multi-objective search with truncated dominance make the same OPEN insertions and return the same solutions.

Proof sketch. Both tests at generation (lines 13–14 of Algorithm 1) are pure—neither modifies a frontier. Their order thus cannot change their conjunction. By Lemma 1, LocalDom is exact. Induction on extractions shows both searches maintain identical frontier coverage and make identical pruning decisions. $\square$

5 Implementation

Adaptive k-d trees. Each $F(s)$ starts as an unindexed array. A frontier is promoted to a tree once it contains ≥ 8 vectors and average query cost exceeds 64 comparisons. Trees are rebuilt when live size exceeds $1.25 \times (\text{size at last build}) + 64$. These parameters affect performance, not correctness.

Residue forest. $R(s)$ is append-only, using the logarithmic method: append new vectors to a buffer; at 64 vectors, promote the buffer to an immutable k-d tree; merge blocks while a predecessor is less than twice a successor's size. Result: $O(\log|R(s)|)$ blocks per state, $O(\log|R(s)|)$ rebuilds per vector.

Test order. At generation (lines 13–14), local dominance is tested before heuristic selection. Both are pure, so order is interchangeable for correctness but affects performance: testing the cheaper, more-selective test first avoids wasteful heuristic evaluations.

6 Experiments

Test problems are grids and road networks. Grids are four-connected $n \times n$ graphs with integer edge costs; pairwise objective correlations range $\rho \in [-0.6, 0]$. Road networks are DIMACS subgraphs (NY, Bay Area). Multi-valued heuristics are backward Pareto sets computed with A*pex. All instances sort heuristics lexicographically.

Machine: Intel Core i5-1135G7, 16 GB RAM, Windows 11. Compiler: GCC 16.2, flags -O2 -DNDEBUG, C++20. Times exclude parsing and output.

We selected four representative instances spanning three to six objectives.

Table 1: Results on representative instances. Single run; all solutions byte-identical.

Instance	M	Solutions	Expansions	Baseline (s)	FAST-MVH (s)	Speedup
Grid 10	3	1,602	9,500	0.058	0.032	1.8×
NY-5	4	7,105	144K	9.55	1.04	9.2×
Grid 8	5	9,412	33K	1.47	0.24	6.1×
NY-3	6	11,286	127K	104	1.27	82×

At three objectives, speedup is modest (1.8×). Truncated queries are short (only 2 coordinates), and Grid 10's frontier remains below tree threshold throughout. The speedup primarily reflects node storage (inline f values).

From four objectives onward, speedup grows significantly (9.2× to 82×). Road instances benefit

more than grids: sparse, high-variance Pareto frontiers make bounding-box pruning selective. NY-3 at six objectives: baseline performs 1,097 million full-dimensional comparisons; FAST-MVH performs 13.3 million—a 82× reduction, matching overall speedup.

Fallback efficacy

Without indexing, FAST-MVH would still scan $R(s)$ linearly. The residue forest’s benefit is entirely from indexing.

Table 2: Full-dimensional fallback reduction. Fallback = queries to $R(s)$. Comparisons = full-vector comparisons in millions.

Instance	M	Fallbacks	Baseline Cmp (M)	FAST-MVH Cmp (M)	Reduction ($\times$)
NY-5	4	65,891	42.8	1.2	35.7 $\times$
NY-3	6	185,455	1,097	13.3	82.4 $\times$

7 Conclusion

Spatial indexing of truncated frontiers and an indexed residue set reduce dominance-checking cost in multi-objective search with multi-

vectors, yet searches are 80–100 $\times$ faster. Speedup comes entirely from indexing, not set size reduction. valued heuristics. Three design choices—k-d trees on truncated frontiers, local-first test order, and a logarithmically decomposed residue forest—achieve 1.8 $\times$ to 82 $\times$ speedups without changing search decisions or solutions.

References

1. Bentley, J. L. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Communications of the ACM* 18(9): 509–517.
2. Bentley, J. L.; and Saxe, J. B. 1980. Decomposable Searching Problems I: Static-to-Dynamic Transformation. *Journal of Algorithms* 1(4): 301–358.
3. Demetrescu, C.; Goldberg, A. V.; and Johnson, D. S., eds. 2009. *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*. DIMACS Series 74. American Mathematical Society.
4. Zhang, H.; Salzman, O.; Kumar, T. K. S.; Hernandez Ulloa, C.; Suazo, L.; and Koenig, S. 2022. A*pex: Efficient Approximate Multi-Objective Search on Graphs. *Proceedings of ICAPS*, 394–403.